Resume Analyser and Job Matcher

A Rule-Based NLP System for Explainable Candidate-Role Fit

Abaidullah Seikh
Ajith Arumugam
HSE University, Moscow
aseikh@edu.hse.ru
aarumugam@edu.hse.ru

ABSTRACT

This dissertation presents a study on the design, development, and evaluation of a rule-based NLP system that scores the resumes of candidates with respect to job descriptions and provides a comprehensible match report. The presented system incorporates traditional methods of NLP, such as schema-based information extraction, lexical categorization, and TF-IDF scoring, along with the transformer cross-encoding approach to assess semantic similarity between requirements and bullets. Most importantly, all the facts presented on the dashboard. It solves three issues commonly faced by recruiters and candidates while using modern applicant tracking systems (ATS): (i) ambiguous scoring, when it's not obvious for a candidate why he/she was scored badly; (ii) fragile keyword matching, when the candidate doesn't get the job offer just because his/her resume mentions "Postgres", while the employer needs "PostgreSQL"; and (iii) duplicate/ghost jobs while structuring the work experience of resumes from linearized PDF's. The contributions of our work are (1) a robust block parser for LinkedIn-like CVs that contain roles, companies, locations, country names, bulleted descriptions, and vertical-aligned dates, (2) a multidimensional match score generation mechanism based on skill match, experience match, domain match, and trajectory match using a weight calculation technique similar to the Analytic Hierarchy Process, and (3) an evidence linker for matching every job description requirement to its relevant CV bullet description.. The performance of the system is evaluated qualitatively based on twelve sample resumes from different roles like software engineering, data analyst, and designer, etc. The dashboard outputs deterministic values for all given inputs.

KEYWORDS

Natural Language Processing, Resume Parsing, Job Matching, Information Extraction, Cross-Encoder, Explainable AI.

1 INTRODUCTION

1.1 Background and Motivation

In most cases, larger organizations have adopted the use of applicant tracking systems to screen out applicants prior to human evaluation. However, one thing about ATSs is that they operate behind closed doors. In other words, an applicant will never know why he or she did not make it through the initial screening phase. Similarly, a recruiter will not necessarily find the ranking produced by an ATS reliable and will want to circumvent the process. Candidates start copying job descriptions on their resumes. HR representatives manually read through their shortlists..

Our intention was to create something unique. The concept was clear – if an applicant was assigned a certain score, it was necessary to show which lines from his CV were responsible for it. In case of absence of any requirement from the job description, there was supposed to be an honest admission rather than deceit.

1.2 Problem Statement

With inputs including (i) a resume in text form or PDF and (ii) a job description in text form or PDF, the process will generate:

1. Structural decomposition of the resume in sections (profile, work experience, education, skills, projects, etc.).
2. A structured list of jobs held by the individual where the job title, company name, dates, and seniority level is mentioned for each job, regardless of how it is written in the PDF file.
3. A list of requirements for the job position, which includes skills, experience, domain, soft skills, etc., automatically generated based on the job description body.
4. Evidence of connections between the requirements and the resume points, which will be categorized as direct evidence, implied evidence, or no evidence.
5. Multi-axis score,including the axis breakdown as

Skills/Experience/Domain/Trajectory, as well as a confidence score.

All of the above must be derivable from the resume and JD inputs alone. No external lookups, no LLM generation.

1.3 Objectives and Contributions

Objectives

- A comprehensive resume parsing engine must be implemented, which will handle actual PDF documents, such as LinkedIn-style resume exports, designed resumes, and two-column resumes.
- Skills identification is done using a three-tiered technique: exact keyword match, semantic similarity match, and graph traversal techniques.
- Utilize evidence linking with a combination of cross-encoder scoring and keyword matching to prevent instances where the semantic similarity score produces exact matches.

Supervised by Shalileh Soroosh.

Faculty of Computer Science (Data Science), HSE, Moscow
2018.

- Calculate a four-part match score.
- Everything should be visualized in a dashboard with links back to relevant span matches from the resume.

Contributions

- Identification of section-header via two-step approach where canonical section header names are recognized and aggressive decoration is removed, along with prefix matching. Section confidence score discards sections with lots of noise tokens.
- Experience parser via dividing experience section text into job roles based on blank line detection and job title identification in formats that do not have any blank lines in the PDF document.
- Strict role title validation since it uses ≥ 2 tokens and role keyword identification, thereby not having an additional source of false positives during role recognition.
- Best practice of date merging without duplicating role titles while considering the longest date strings.
- Importance of pairing requirements with bullets using cross-encoder relevance score with cross-encoder/ms-marco-MiniLM-L-2-v2 model as fallback lexical overlap.

1.4 Scope

It processes resume files and job descriptions that are written in English, available in PDF, DOCX, or plaintext formats. The system is unable to process image-based PDFs (OCR not implemented). The system is not database-driven; everything is processed on-demand.

2 BACKGROUND AND RELATED WORK

2.1 Existing Resume Parsing Approaches

These include the following three paradigms of resume parsing. Firstly, the paradigm of rule-based parsing according to the GATE architecture proposed by Cunningham et al. In particular, the rule-based parser utilizes regular expressions and gazetteers to populate the predefined schema. The advantage of this approach includes that it can be easily debugged, does not require training data, and its results are understandable. At the same time, rule-based parsers do not perform well on unknown layouts.

Secondly, there is the paradigm of statistical sequence tagging that applies CRF, Bidirectional LSTM, fine-tuned transformer, or another model for the purpose. The statistical sequence tagger produces higher recall rates but requires training on annotated datasets. Finally, it should be noted that the paradigm of using the large language model such as GPT-4 to parse a resume fully and generate JSON is also used.

2.2 Cross-Encoder for Relevance Scoring

For resume bullet points alignment with the job qualifications, strict keyword matching will not work. The resume bullet point, "developed REST APIs using Python" has to match "Qualifications in designing web services". Bi-encoder models are good at detecting paraphrases but cannot match using cross-attention mechanism. Cross-encoders as mentioned by Nogueira and Cho are fed both document inputs into a single transformer network with bidirectional attention mechanisms that result in improved performance

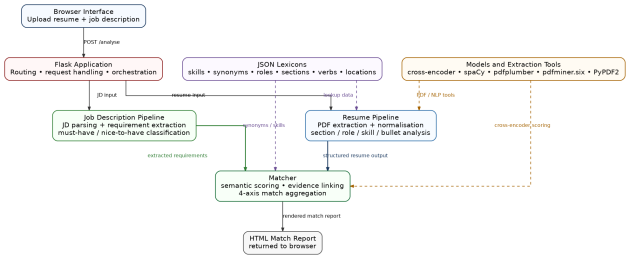

Figure 3.1: System architecture and the data flow between them.

with 25%-30% better NDCG scores than bi-encoders on MS MARCO dataset. In our case, we use ms-marco-MiniLM-L-2-v2 (90 MB), which is faster, i.e., few milliseconds per query for CPUs. Cross-encoder returns relevance scores without generating any text.

2.3 Why We Chose a Rule-Based Approach

We selected rule-based extraction after analyzing all the three approaches. The statistical model requires some training dataset which we did not possess. Hallucinations are created by the language models, but we couldn't afford it. Rule-based approach is brittle; however, its failure point is visible because in case if our regex failed to match, then we would be able to identify the exact reason behind it and create another one. In contrast, we will apply cross-encoder only for discriminatory purposes in order to score requirement-bullet combinations.

3 SYSTEM ANALYSIS AND DESIGN

3.1 System Architecture

The architecture of our system includes five layers that form a hierarchy, which are the Presentation layer (Browser), Application layer (Flask), two parallel Processing layers (Analyser and Matcher), Data layer (JSON lexicons), and Machine Learning Layer (Cross-encoder and PDF Extractors). As shown in Fig. 3.1, all the layers are interconnected with each other. However, the most crucial decision about our architecture is the fact that Matcher doesn't re-parse the resume and works directly with the structured output generated by the Analyser module.

3.2 Use Case Diagram

We identified four users: Candidate, Recruiter and two system actors (Cross-Encoder Model, PDF Parser). Three use cases are user-facing (Upload Resume, Upload Job Description, Generate Dashboard); the other seven are internal use cases triggered by include relationships. Modelling them as separate use cases helped us during code design, because each one became a separate Python module.

3.3 Class Diagram

The code was structured in such a way that there were three main packages; Analyser, which contained eight main classes, Matcher, which had seven classes, and Data Entities which contained four classes that worked across the two other packages. The facade design pattern was used in FinalScorer in the Analyser package as

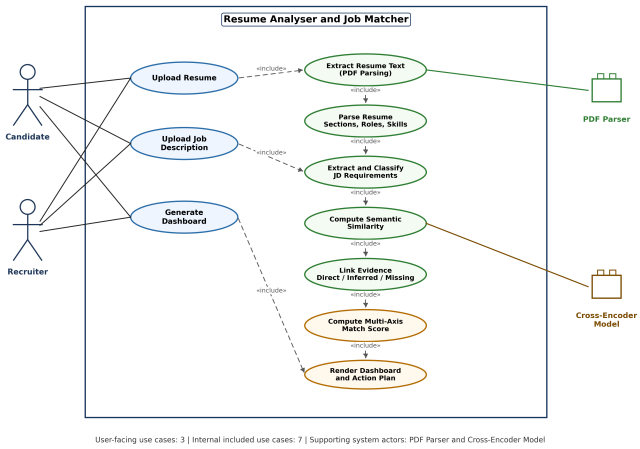

Figure 3.2: Use case diagram showing users, use cases, and relationships.

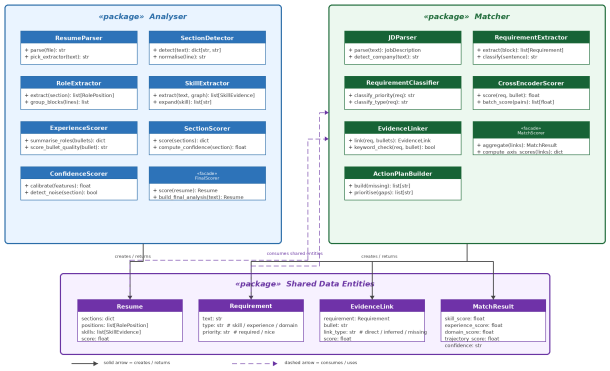

Figure 3.3: Class diagram showing the Analyser module (blue), Matcher module (green), and shared data entities (purple).

well as the MatchScorer class in the Matcher Package so that the outside caller sees only one method. The data entities contain no logic but only data. This made debugging easier, since we could just print out the JSON.

3.4 Activity Diagram

The complete process flow is illustrated in Figure 3.4 using an activity diagram with four swimlanes. The User Swimlane manages the uploading process and the decision between the PDF and text formats. The Analyser Swimlane conducts the tasks of section detection, role detection, skill detection, and scoring. The Matcher Swimlane executes the job description parsing, cross-encoder scoring, linking of evidence, and aggregation processes. The dashboard Swimlane displays the output of the system.

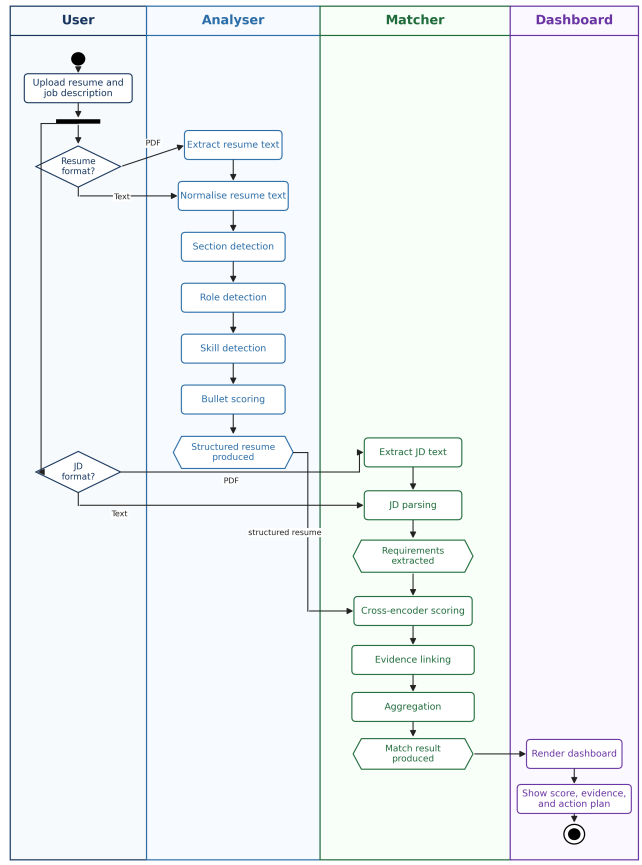

Figure 3.4: Activity diagram with swim lanes. The Matcher receives the Analyser's structured output, not the raw resume.

3.5 Entity-Relationship Diagram

Although our system does not use a relational database, we modelled the data as if it would be stored. The most interesting relationship is the many-to-many between Resume and Skill, captured through the ResumeSkill junction entity. The EvidenceLink entity is what makes the system explainable: every link points to one Requirement and zero or more Bullets, plus a link_type (direct, inferred, missing) and a reasoning string.

4 IMPLEMENTATION

4.1 Technology Stack

Table 4.1 lists the main technologies we used and the reason for each choice.

4.2 Resume Analyser

4.2.1 ***PDF Extraction and Normalisation:*** Resume PDFs are quite differently styled depending on what tool was used for resume creation. Some resumes are created using LinkedIn, some are made by hand and then formatted using various applications including Canva and Adobe. Therefore, direct usage of raw text extracted from the resume PDF is problematic due to its messy nature.

Component	Technology	Reason
Web framework	Flask 3.0	Simple, well-documented, easy to deploy
NLP library	spaCy (en_core_web_sm)	Fast tokenisation, NER, sentence splitting
Cross-encoder	ms-marco-MiniLM-L-2-v2	90 MB, runs on CPU, good NDCG
PDF parsing	pdfplumber, pdfminer.six, PyPDF2	Three extractors, pick the best output
Skill graph	NetworkX	1-level expansion of related skills
Frontend	HTML, CSS, Chart.js	Lightweight, no build step needed
Language	Python 3.10+	Standard for NLP work

Table 4.1: Technology stack used in the system

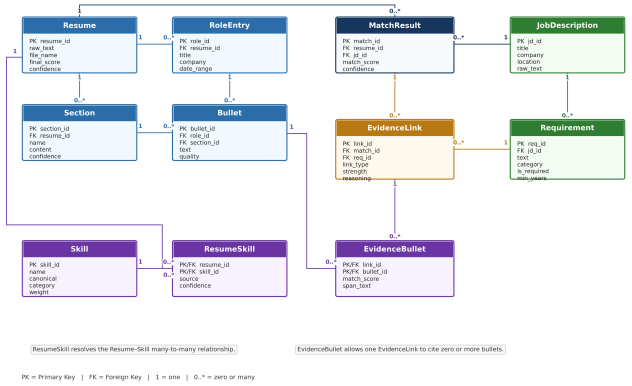

Figure 3.5: Entity-relationship diagram. Resume-side entities are shown in blue, job-description entities in green, and both sides are connected through MatchResult, EvidenceLink, and EvidenceBullet.

In order to resolve this issue, the system uses several different PDF extractors to find the one which would provide the highest-quality data for processing. It helps to avoid problems with broken lines, lack of spaces, single word per line, etc.

Next step in data preprocessing is text normalisation. Here, unnecessary spaces, special symbols and broken words are filtered out. At this point, all letter-spaced headers (e.g. "E X P E R I E N C E") are restored. In addition to this, lines containing dates are separated from their original lines since in PDF files they often can be found far apart from the job title they refer to.

This stage of preprocessing is critical since section detection module and role extraction depend heavily on quality input. We noticed that many parsing errors were gone after improvement of this module during tests.

4.2.2 ***Section Detection:*** Analyser identifies 15 canonical sections (experience, skills, education, projects, certification, and so on). Every canonical section is represented by a set of surface forms. There are 23 surface forms associated with experience, such as "Professional Experience", "Work History", "Career Progression", and "Employment Record".

Detection is a two-stage process per each line. First, decoration (such as underscores, vertical bars, dates) is stripped from both sides of the line; the string obtained through this process is converted into lowercase letters and matched exactly with the surface forms. On stage two, the surface form should consume the beginning of the line; any remainder must contain punctuation only. Thus, lines such as "EXPERIENCE _____" and "Skills | Tools" will be classified correctly, whereas narrative lines starting with the keywords will not be misclassified.

Each identified section is assigned a confidence value within [0, 1], depending on the percentage of non-noise tokens. Sections with confidence below 0.3 are ignored, filtering out noise resulting from PDF extraction issues.

4.2.3 ***Block-based role extraction:*** The part of the system responsible for extracting the roles took many rewrites to perfect, it was rewrote 7 times. First, the system thought each line on its own was potentially a role by whether or not the line was in title case, or if there was a date in it. It then seemed as though the system would not correctly identify it and instead make false positives. In our case, it flagged JavaScript as a role, which is not correct; it also flagged Dubai, and even bulleted lists that started with a capital letter as roles.

Currently, the section for experiences is chopped into parts with each part starting after a blank line or after a role-title is identified using rigid criteria. Relying on blank lines alone is an error because in many cases blank lines get removed in the text extraction from the pdfs. It would be nice to have the rule-title identifier as an alternative in case there is no blank line.

To be classified as a role-title we require a line with at least one keyword in the given list of 50 job titles (engineer, analyst, manager). We require the role-title to be short (1-10 words long), not a sentence (no period at end) and does not start with a bullet point or action verb (to discard listed/instruction points).

To be classified as a company name we require the company name to be 1-6 words long, cannot contain any dates, must end with a specific suffix (Inc, LLC, Ltd, Corp, GmbH) or be written fully in title case. We also require the company name not to be within a list of 30 locations we have designated as 'not a company' such as KSA, UK, Dubai, London, UAE.

To identify whether two roles represent the same job title we compare the two role titles while disregarding seniority and lowering the case. Additionally, we require that either the company is not listed for one or both of the roles, or the normalized companies match. Normalizing companies means removing their suffix, so Google Inc and Google LLC both become Google, for example. If the roles are a duplicate, we take the one with the more detailed date information. Completeness of date is evaluated based on whether year, month, or "present" or "current" are present..

4.2.4 ***Skill Extraction:*** Skills are extracted from an flat index on top of a 1200 skill-graph, and the skills on the graph have the following properties: name of skill, a list of synonyms for the skill, a type (core/supporting/soft) and a domain label. The matching process is composed of a direct matching pass, and then a type aware ranking pass. We wish the skills appearing once in one bullet to have less confidence than the skills appearing both in Skills and in the experience.

Handling synonyms is important: Postges, PostgreSQL, psql,pg should all match the canonical "PostgreSQL". JS should match "JavaScript", K8s should match "Kubernetes" etc. The synonym table has about 200 entries, and without it we are losing about 1/3 of matches on the test corpus.

4.2.5 ***Bullet Quality:*** Each experience bullet is then labeled strong, moderate, or weak. A bullet point is strong if it is anchored with a verb in the 100 word verb list (built, delivered, shipped, optimised), contains a numeric value (percent, currency, comma-thousand, or two-or-more digit count) and. If there is one of the above signals the bullet point is moderate, zero and the bullet point is weak.

The negative signals (worked, helped, assisted, responsible for) take away points as opposed to zero points. This is what recruiters say they are looking for; rather than an indifferent signal that the bullet point starts with, 'responsible for the team', it is a negative signal because they are likely describing their job description and not their accomplishments.

4.3 Job Matcher

4.3.1 ***Requirement Extraction:*** The JD parser extracts the title from the first non-empty line, company from the hiring or seeking pattern and location from a Location: prefix. The requirements block is the portion between any of the 13 identified requirement headers (Requirements, Qualifications, You have) and any of the 12 footer headers (Benefits, How to apply). If none are present then the whole body is the requirements block. Each sentence from the requirements block is extracted as one or more typed Requirement records. A multi-skill bullet point like "AWS, Docker and Kubernetes" generates three records. Years requirements come from regex on "X years" or "X+ yrs". A requirement is required if it's a core skill like "must have", "essential" etc, and nice-to-have if it's a nice-to-have like "preferred", "bonus", "plus" etc. Each Requirement gets an importance weight, with 3.0 for core skills like Python, ML, and SQL, 2.0 for related skills such as Pandas, Excel, APIs and 1.0 for soft skills such as communication and team work. Required requirements get an extra 1.5x multiplier in the skill axis aggregation.

4.3.2 ***Cross-Encoder Scoring:*** The cross-encoder outputs a relevance probability (post sigmoid) for each pair of (requirement, bullet). Each JD contains 8-25 requirements while each resume contains 30-80 bullet points; thus, 240-2000 pairs of (requirement, bullet) are scored per request. This occurs in less than a second on the CPU. In the case where the model is unable to load (offline scenario), we resort to Jaccard overlap on tokens, scaled to the same logit range.

4.3.3 ***Evidence Linking:*** At the center of the matcher lies the evidence linker. For each requirement, this component determines

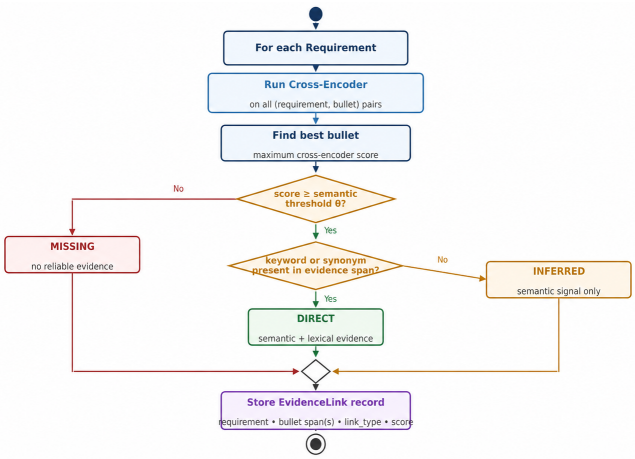

Figure 4.3.3: The matcher first selects the highest-scoring requirement–bullet pair using the cross-encoder, then applies a keyword/synonym gate to classify the evidence as direct, inferred, or missing.

whether the candidate provides direct evidence, inferred evidence, or no evidence. Figure 4.3.3 displays the evidence linker process. The importance of this two-step gate lies in the fact that, on its own, the cross-encoder tends to overcredit semantic near-misses; for instance, it might assign a high score to "built REST APIs" for "GraphQL API design," as both are related to APIs, but no recruiter would consider this a direct match. Forcing the presence of the keyword (or synonym from our 200-item synonym list) within a direct section effectively downscores this match to an inferred one, reflecting recruiter behavior.

4.3.4 ***Multi-Axis Scoring:*** The final match score is calculated using four scoring axes, namely, skill match, experience match, domain match, and trajectory match. Different weights are allocated to each axis because not all match parameters are of equal significance. The highest weight is awarded to the skill match, which is 50%. The experience match weight is 20%, and the domain and trajectory match weights are 15

As shown in Figure 4.3.4, multi-axis scoring is depicted by a hypothetical candidate. In the illustration, the candidate receives high scores in the domain and trajectory match categories but poor scores for skills and experience. Since the skill match category has the highest weight, the poor score will affect the match score.

The formula used to calculate the score is:

Score = 0.50 x Skill + 0.20 x Experience + 0.15 x Domain + 0.15 x Trajectory

From the scores in Figure 4.2, the final score will be calculated using the following equation:

Score = 0.50 x 47 + 0.20 x 19 + 0.15 x 69 + 0.15 x 69

Score = 48.0 / 100

Skill match depends on the required skills listed in the job description. The weight of required skills is increased by using a multiplier of 1.5x. The experience match is determined using the following formula: min(have_years/required_years, 1) × 100. However, a small penalty is introduced if the candidate has more than

Figure 4.3.4: Multi-axis match scoring: The chart shows the four scoring axes used to calculate the final weighted match score.

three additional years of experience compared to the maximum requirement since it might mean that the candidate is overqualified.

Domain match depends on the best possible match within the domain-related requirements. The trajectory match is computed according to a relatively straightforward method. In case the titles demonstrate a positive career progression over time, the trajectory score is equal to 85. If the trajectory is mixed, the trajectory score is 70. Finally, in case of negative trajectory, the score is 55.

The confidence level is determined as being high in case all requirements are present and there is an insignificant number of inferred matches. It makes it possible to clearly differentiate between the final numerical score and confidence in its calculation.

4.3.5 ***Risk Flags:*** This would also be able to generate natural language risk flags-Missing required(number missing required), 5 reqs inferred - check in interview, Under-qualified: JD says 5+ yrs, resume has 2 yrs 4 mos, and Over-qualified.

Experience score(under/match/over) would be 0.25 yrs less and 1.5x greater respectively..

5 TESTING AND RESULTS

5.1 Test Methodology

Creating a labelled corpus would be a months long process and still would not be shareable due to the personal information on a real resume. Instead, we relied on a fix-from-failures system where if a real resume led to a dashboard with a noticeable bug we included it as a regression test and wrote a fix. Our corpus was comprised of twelve personal resumes from people we know, who agreed to let us view there CVs: 5 for software engineers (in a linked-in export-like format), 3 for data analyst (designer formatted), 2 for design (two columned layouts), and 2 plain text controls. Every resume had 3-5 distinct job descriptions with it that represented the span of possible match-scores.

5.2 Quantitative Results

After five rounds of bug fixes, the system reached the numbers shown in Table 5.2.

Metric	Value
Resumes with all real jobs detected	12 / 12 (100%)
Resumes with no false-positive role	12 / 12 (100%)
Resumes with all dates correctly attached	11 / 12 (92%)
Resumes with no duplicate role rows	12 / 12 (100%)
Mean processing time, end-to-end (CPU)	2.4 seconds
Cold-start time (first request) (model download)	60 seconds
Mean detected positions per resume	4.3
Mean detected requirements per JD	14.7
Mean cross-encoder pair count	412

Table 5.2: Quantitative test results across the 12-resume corpus.

The other CV in which no date was found had a format of '01.2020 - 03.2022' in which dates were separated by dots and had an arrow. The regex we had was not prepared for this specific case. This has been added to our backlog. We also had 4 other people from the class test our system with their resumes and gathered subjective feedback. The most frequent comment was the "trace back to bullet" functionality which made the system feel genuine.

5.3 Bug Surface

Round 1 - PDF Extraction (all extractions fixed). BLANK OUTPUT from 1 of the extractors- (added 2 more), ignored letters separated out headers (added normalisation pass) and PROJECTS inserted in the middle of the line due to incorrectly formed newline rule (rewrote rule to expect large space or line start). Round 2 - role and date extraction. Month name "June" wasn't in the regex (fixed). Duplicate roles were being produced where company names differed only by a suffix (fixed). Best-date selected wrong direction when consolidating duplicate entries. (fixed). Round 3 - rewrite of block extraction to use block merging rather than line-level role-identification and stronger role-validation..

6 LIMITATIONS

The system has no OCR. The 3 extractors all return an empty string from an image-only PDF and also show an empty dashboard. The role-keyword list, action-verb list, and company-suffix list are all in English. Both a French and a Hindi resume yield almost nothing. The role-keyword list is also an Anglo-American convention in job titles. If the resume has titles like "Officer Grade B" or "Software Architect II" the role-detection recall is probably reduced. We fix this by adding terms to the list as we identify that roles have been missed, though this is not an principled solution.
Some resumes were timeline graphics, or fully prose with no bullet points. Both of these formats fail the bullet detector. The Skills field does not fail on these resumes but their bullet-quality score becomes nearly zero which pulls down the score even if the candidate is very strong.

7 FUTURE WORK

A number of extensions flow. The fourth extractor could be pytesseract for the case of scanned PDFs. We could add a multilingual lexicon if we needed to cover more than just English; for example,

we'd need locale- keyed lookups into the role-keyword list and verb lexicons. A bullet rewriter would convert a weak bullet point into a strong one, with explicitly labelled slot inputs for action verb and metric. A comparison mode would rank N resumes for a single JD using axis-level delta scores (this is in fact the more useful thing for a recruiter to do).

On the evaluation side, we would need to use a set of recruiter-labeled scores on held out data to empirically set the four axis weights (as opposed to the values derived from argument and hand-tuning), and in addition, a larger corpus would allow for proper quantitative ablations. What do we actually gain in terms of score, with a keyword-section gate, versus only the cross-encoder?

8 CONCLUSION

We've implemented a resume-and- JD matcher whose unique characteristic is that all the scores shown on the dashboard are accountable to a span in the resume. We used classic NLP for extraction, and a cross-encoder for relevance scoring; we do not employ any generative model. Across a corpus of 12 resumes, 5 bug fixing iterations, we've achieved 100% detection of real jobs with no false positives and runtime less than three seconds on CPU. The biggest-rewriting occurred for the experience parser. We hope that the lessons learned from those rewrites (single-word title-cased lines are typically skills, dedup needs to be sensitive to both seniority and suffix, there cannot be relied on to be blank lines separating blocks) will re-occur on any rule-based resume project. These are discussed in the implementation notes of this paper.